



Coding Guideline v02 01

Kinh doanh quốc tế (Asia International Academy University)



messages.pdf_cover_qr_code_label

ISO26262 Coding Guideline

**Prepared For:
SW Development**

Table of Content

TABLE OF CONTENT	2
1. INTRODUCTION.....	6
1.1. Purpose of the document	6
1.2. Reference	6
2. NAMING CONVENTION.....	8
2.1. File Naming	8
2.1.1. Name_File_001	8
2.1.2. Name_File_002	8
2.2. Variable Naming.....	8
2.2.1. Name_Var_001	8
2.2.2. Name_Var_002.....	10
2.2.3. Name_Var_003.....	10
2.2.4. Name_Var_004.....	10
2.3. Function Naming.....	11
2.3.1. Name_Func_001	11
2.4. Type Naming.....	11
2.4.1. Name_Type_001	11
2.4.2. Name_Type_002	11
2.5. Macro Naming	12
2.5.1. Name_Macro_001.....	12
3. CODING STYLE.....	13
3.1. Format	13
3.1.1. Style_Format_001	13
3.1.2. Style_Format_002	13
3.1.3. Style_Format_003	15
3.1.4. Style_Format_004	15
3.1.5. Style_Format_005	15
3.1.6. Style_Format_006	15
3.1.7. Style_Format_007	16
3.1.8. Style_Format_008	16
3.1.9. Style_Format_009	16
3.1.10. Style_Format_010.....	17
3.1.11. Style_Format_011.....	17
3.1.12. Style_Format_012.....	17
3.1.13. Style_Format_013.....	18
3.1.14. Style_Format_014.....	19
3.1.15. Style_Format_015.....	19
3.1.16. Style_Format_016.....	19
3.1.17. Style_Format_017.....	20
3.2. File Structure.....	20
3.2.1. Style_File_001	20
3.2.2. Style_File_002	20
3.2.3. Style_File_003	21
3.2.4. Style_File_004	21
3.2.5. Style_File_005	21
3.2.6. Style_File_006	22
3.3. Comments.....	22
3.3.1. Style_Comment_001	22
3.3.2. Style_Comment_002	22
3.3.3. Style_Comment_003	23
3.3.4. Style_Comment_004	23

3.3.5. Style_Comment_005	23
3.3.6. Style_Comment_006	24
3.3.7. Style_Comment_007	24
4. CODING RULES.....	25
4.1. Conformance to external rules.....	25
4.1.1. Rules_Ext_001	25
4.1.2. Rules_Ext_002	25
4.2. Environment	25
4.2.1. Rules_Envr_001	25
4.2.2. Rules_Envr_002	25
4.2.3. Rules_Envr_003	26
4.2.4. Rules_Envr_004	26
4.2.5. Rules_Envr_005	26
4.2.6. Rules_Envr_006	27
4.3. Comments/Documentation.....	27
4.3.1. Rules_Comment_001.....	27
4.3.2. Rules_Comment_002.....	27
4.3.3. Rules_Comment_003.....	27
4.3.4. Rules_Comment_004.....	28
4.3.5. Rules_Comment_005.....	28
4.3.6. Rules_Comment_006.....	28
4.4. Identifiers	29
4.4.1. Rules_Identifier_001	29
4.5. Types.....	29
4.5.1. Rules_Types_001	29
4.5.2. Rules_Types_002	30
4.5.3. Rules_Types_003	30
4.5.4. Rules_Types_004	30
4.6. Constants.....	31
4.6.1. Rules_Const_001.....	31
4.6.2. Rules_Const_002.....	31
4.6.3. Rules_Const_003.....	31
4.7. Declarations and Definitions.....	31
4.7.1. Rules_Defn_Decl_001	31
4.7.2. Rules_Defn_Decl_002	32
4.7.3. Rules_Defn_Decl_003	32
4.7.4. Rules_Defn_Decl_004	32
4.7.5. Rules_Defn_Decl_005	33
4.7.6. Rules_Defn_Decl_006	33
4.7.7. Rules_Defn_Decl_007	34
4.7.8. Rules_Defn_Decl_008	34
4.7.9. Rules_Defn_Decl_009	34
4.7.10. Rules_Defn_Decl_010.....	35
4.7.11. Rules_Defn_Decl_011.....	35
4.7.12. Rules_Defn_Decl_012.....	35
4.7.13. Rules_Defn_Decl_013.....	35
4.7.14. Rules_Defn_Decl_014.....	36
4.7.15. Rules_Defn_Decl_015.....	36
4.7.16. Rules_Defn_Decl_016.....	36
4.7.17. Rules_Defn_Decl_017.....	36
4.7.18. Rules_Defn_Decl_018.....	37
4.7.19. Rules_Defn_Decl_019.....	37
4.7.20. Rules_Defn_Decl_020.....	37
4.7.21. Rules_Defn_Decl_021.....	38
4.7.22. Rules_Defn_Decl_022.....	38

4.7.23. Rules_Defn_Decl_023	38
4.7.24. Rules_Defn_Decl_024	39
4.7.25. Rules_Defn_Decl_025	40
4.7.26. Rules_Defn_Decl_026	40
4.8. Initialization	40
4.8.1. Rules_Init_001	40
4.9. Control Statement Expressions	41
4.9.1. Rules_Expr_001	41
4.9.2. Rules_Expr_002	41
4.9.3. Rules_Expr_003	41
4.9.4. Rules_Expr_004	42
4.9.5. Rules_Expr_005	42
4.9.6. Rules_Expr_006	42
4.9.7. Rules_Expr_007	42
4.9.8. Rules_Expr_008	43
4.9.9. Rules_Expr_009	43
4.9.10. Rules_Expr_010	43
4.9.11. Rules_Expr_011	44
4.9.12. Rules_Expr_012	44
4.9.13. Rules_Expr_013	45
4.10. Control Flow	45
4.10.1. Rules_CtrlFlow_001	45
4.10.2. Rules_CtrlFlow_002	46
4.10.3. Rules_CtrlFlow_003	46
4.10.4. Rules_CtrlFlow_004	46
4.10.5. Rules_CtrlFlow_005	47
4.10.6. Rules_CtrlFlow_006	47
4.10.7. Rules_CtrlFlow_007	48
4.10.8. Rules_CtrlFlow_008	48
4.10.9. Rules_CtrlFlow_009	48
4.10.10. Rules_CtrlFlow_010	49
4.10.11. Rules_CtrlFlow_011	50
4.10.12. Rules_CtrlFlow_012	50
4.10.13. Rules_CtrlFlow_013	50
4.11. Functions	51
4.11.1. Rules_Func_001	51
4.11.2. Rules_Func_002	51
4.11.3. Rules_Func_003	51
4.11.4. Rules_Func_004	51
4.11.5. Rules_Func_005	52
4.11.6. Rules_Func_006	52
4.11.7. Rules_Func_007	54
4.12. Pointers and Arrays	54
4.12.1. Rules_Ptr_001	54
4.12.2. Rules_Ptr_002	54
4.12.3. Rules_Ptr_003	55
4.12.4. Rules_Ptr_004	55
4.12.5. Rules_Ptr_005	55
4.13. Structures and Unions	56
4.13.1. Rules_Struct_001	56
4.14. Pre-processing Directives	56
4.14.1. Rules_PreProcess_001	56
4.14.2. Rules_PreProcess_002	56
4.14.3. Rules_PreProcess_003	57
4.14.4. Rules_PreProcess_004	57

4.14.5. Rules_PreProcess_005	58
4.14.6. Rules_PreProcess_006	58
4.14.7. Rules_PreProcess_007	58
4.14.8. Rules_PreProcess_008	59
4.14.9. Rules_PreProcess_009	59
4.14.10. Rules_PreProcess_010	59
4.14.11. Rules_PreProcess_011	60
4.14.12. Rules_PreProcess_012	60
4.15. Optimization	60
4.15.1. Rules_Opt_001	60
4.15.2. Rules_Opt_002	60
4.15.3. Rules_Opt_003	61
4.15.4. Rules_Opt_004	61
4.15.5. Rules_Opt_005	62
4.15.6. Rules_Opt_006	62
4.15.7. Rules_Opt_007	62
4.15.8. Rules_Opt_008	62

1. Introduction

1.1. Purpose of the document

This is an example of coding guideline for ISO26262 development.

The purpose of this document is to lay down the C coding rules, style and naming convention which has to be followed for development of SW components during the software development in C language.

The C coding guidelines gives the following benefit:

- It improves the software quality, readability and maintainability.
- It minimizes the software bug.

1.2. Reference

[1] ISO26262 Standard First Edition 2011 - Part 6:

“Part 6_Product development at the software level.pdf”

[2] ISO26262 Standard - Part 6 - Product Development at software level

“2018_2nd_ISO26262-06.pdf”

Revision History

#	Version	Date	Reason for change	Approver	Checker	Author
1	1.00	31-Jan-2020	Support ISO 26262:2018 mapping	Y.Inoue	-	Nhu Nguyen
2	2.00	30-June-2020	1) Revised ISO requirement of ISO 26262:2018 mapping of all rules base on new analysis. 2) Remove all mapping of ISO 26262:2011 3) Revise content of following rule base on new analysis to improve quality: + Add or improve examples for the following rules for better understanding: Style_Format_016, Style_File_002, Style_File_006, Rules_Comment_001, Rules_Expr_005, Rules_Expr_006, and Rules_PreProcess_004. + Rule Name_Var_003: Fix the typo “meaning full” to “meaningful”. + Rule Name_Var_001 and Name_Var_002: Give the format example into description for easy to understand. + Rule Rules_PreProcess_010, Rules_PreProcess_011 and Rules_CtrlFlow_010: Replace “¥” by “\” (yen by backslash) for international reader. + Rule Name_Type_001: Remove the redundant example in content of description. + Rule Style_Format_005: Increase the limited columns from 80 to 120 by considering recent conventions. + Rule Rules_Comment_003 and Rules_Defn_Decl_024: Improve the description and add examples for easy to	Y.Inoue	Hibiki Nano, Wei Liu	Duy Do Hy Nguyen

			understand. + Rule Rules_Expr_002: Decrease the control structure nesting from 6 levels to 4 levels in line with RCT-JB5001 criteria (QA data). + Rule Rules_CtrlFlow_003: Make clear criteria for cyclomatic complexity numbers in line with RCT-JB5001 criteria (QA data) i.e. less than 30 + Rule Rules_CtrlFlow_008, Rules_Func_003 and Rules_Ptr_001: Improve the description for easy to understand. 4) Revise format of "Revision history" table and add "Checker" column.			
3	2.01	30-Sep-2020	Revise the content of Rules_CtrlFlow_003 rule by replacing "commented the top of a file" to "documented".	Y.Inoue	Hibiki Nano, Wei Liu	Hy Nguyen

2. Naming Convention

This section specifies the naming conventions to be followed for file, functions, variables, constants, macros and types.

2.1. File Naming

2.1.1. Name_File_001

File name shall only consist of alphanumeric characters and underscore. But it shall not start with underscore or numeric.

Example:

```
/* Compliant */  
Adc.c  
Wdg_59_DriverA.c
```

```
/* Not compliant */  
_Adc.c  
59_Spi_Driver.c
```

Rationale:

Even though most operating systems nowadays can handle spaces and special characters in file names some of the tool used in the software development may not handle them properly.

2.1.2. Name_File_002

File name shall not contain more than 31 characters.

Example:

```
/* Compliant */  
Icu_Reg_eMIOS_LLD_CfgEx.h (25 characters long)
```

```
/*Not compliant */  
Icu_Register_IPV_eMIOS_LLD_CfgEx.h (34 characters long)
```

Rationale:

Even though most operating systems nowadays can handle very long file name however some of the tool used in the software development may not handle them properly.

2.2. Variable Naming

2.2.1. Name_Var_001

All SOFTWARE Modules shall label local variables according to the following scheme:
Composition of name: <Variable name>

Spelling of name: First letter of each word upper case, consecutive letters lower case.

The variable name should start with 'L' followed by 2 characters for data type and then the variable name. Example format: L<data type><variable name>

The table 1 lists all common data types and the corresponding character string, which should be used in variable names:

Table 1: Data Types

Keyword	Data Type
dd	derived data type, <Type>_least
uc	unsigned char
sc	signed char
us	unsigned short
ss	signed short
si	signed int
ui	unsigned int
ul	unsigned long
sl	signed long
fl	float
db	double
st	structure
un	union
aa	array
p	pointer
pp	pointer to pointer
en	enumerated type
S Tag	Tag for the structure, followed by the string as specified for the type definition. S Tag_<Component name>_<TypeName> E.g., S Tag_CanIf_Hth
U Tag	Tag for the union, followed by the string as specified for the type definition. U Tag_<Component name>_<TypeName> E.g., U Tag_CanIf_Hth
E Tag	Tag for the enum, followed by the string as specified for the type definition. E Tag_<Component name>_<TypeName> E.g., E Tag_Can_StateTransitionType

Example:

a) local/static variable of type uint32

```
static uint32 LulVarX;
```

b) auto/function scope variable of type uint8

```
uint8 LucResult;
```

c) local/static pointer to type uint32

```
static uint32 *LpPtrName;
```

Rationale:

The variable scope, its owner and its type can be understood from its name.

2.2.2. Name_Var_002

All SOFTWARE Modules shall label global variables according to the following scheme:
Composition of name: <Component name>_<Variable name>

Only one underscore between component name and variable name should be present.
Spelling of name: First letter of each word upper case, consecutive letters lower case.
The variable name should start with G followed by 2 digits for data type and then the meaningful name. Example format: <Component Name>_G<data type><variable name>

The table 1 list all common data types and the corresponding character string, which should be used in variable names mentioned in local variable section holds good for global variables also.

Example:

Global/exported variable of type unit16

/ Compliant */*

```
extern unit16 Adc_GusResult;
```

/ Not compliant */*

```
extern unit16 AdcGusResult;
```

Rationale:

The variable scope, its owner and its type can be understood from its name.

2.2.3. Name_Var_003

A meaningful name (with respect to that context) should be given to the variables. Avoid giving name as Temp, Busy, etc.

Example:

/ Compliant */*

```
uint8 LucTempBufferValue
```

Rationale:

Readability

2.2.4. Name_Var_004

Reserved identifiers, macros and functions in the standard libraries, shall not be defined, redefined or undefined. The names of macros, objects and functions in the standard libraries shall not be reused.

Example:

Not required.

Rationale:

There are some specific reserved words and function names which are known to give rise to undefined behaviours if they are redefined or undefined.

2.3. Function Naming

2.3.1. Name_Func_001

Global function names are structured with upper case/lower case and the first letter should be upper case.

Composition of name: <Component name>_<Function name>

Only one underscore between component name and function name.

Example:

`/* Compliant */`

`Com_NormalReset`

`/* Not compliant */`

`com_NormalReset`

`Com_normalReset`

`Com_Normalreset`

Rationale:

The component to which the function belongs can be understood from its name.

2.4. Type Naming

2.4.1. Name_Type_001

Enumeration elements shall be uppercase, with words separated by the underscore character and with 'ETag' as prefix.

Keyword	Data Type
ETag	Tag for the enum, followed by the string as specified for the type definition. ETag_<Component name>_<TypeName>

Example:

`/* Compliant */`

```
typedef enum ETag_Spi_StatusType
{
    SPI_UNINIT,
    SPI_IDLE,
    SPI_BUSY
} Spi_StatusType;
```

Rationale:

Readability

2.4.2. Name_Type_002

All SOFTWARE Modules shall label self-defined i.e. specific data types according to the following scheme:

Composition of type: <Component name>_<Type name>Type

Only one underscore between component name and type name.
Spelling of type: First letter of each word upper case, consecutive letters lower case.

Example:

```
/* Compliant */  
Eep_LengthType  
Dio_SignalType  
Nm_StatusType
```

Rationale:

Readability

2.5. Macro Naming

2.5.1. Name_Macro_001

Constants defined as a macro shall be written in upper case. Digits and underscores are allowed but not at the start.

Example:

```
/* Compliant */  
#define NR_OF_ELEMENTS    10  
  
/* Not compliant */  
#define nr_of_elements    10
```

Rationale:

Readability

3. Coding style

3.1. Format

3.1.1. Style_Format_001

Each file shall have a file template and file description banner agreed for each project. The template should be defined at the starting of development.

Example:

Not required.

Rationale:

This helps users determine which platforms this code had been developed on and also which version they are currently using.

3.1.2. Style_Format_002

Items of similar purpose such as file inclusion, declaring global variables, function prototypes, etc. shall be grouped together at the top of a source file (or within a header file). The order of these sections shall be:

- For a C source file:

1. File inclusion (system includes - <xxx.h>, followed by project includes - "xxx.h").
2. Macro.
3. Typedef.
4. Exported global.
5. Private global variables and functions.

- For a C header file:

1. Macro definitions.
2. Typedef definitions.
3. Exported global variables (extern).
4. Exported global functions.

Example:

```
1)
/* Compliant */
/* This is an example of the top of a source file */

/*****
Includes <System Includes>, "Project Includes"
*****/
#include <stdio.h>
#include <stdint.h>
#include "spi_driver.h"

/*****
Macro definitions
*****/
#define MAX_PAWS (4)

/*****
```

```

Typedef definitions
*****/
typedef uint16_t doggy_t;
typedef struct
{
    uint8_t paws[MAX_PAWS];
    uint8_t tail;
    uint8_t nose;
} neko_t;

/*****
Exported global variables (to be accessed by other files)
*****/
uint8_t g_example_Distance;
const char g_example_ReplaceText [] = "TEST";

/*****
Private global variables and functions
*****/
static uint8_t wait_time = 0;
static const char end_msg[] = "all over";
static void do_work(void);

```

2)

```

/* Compliant */
/* This is an example of the top of a header file */

/*****
Macro definitions
*****/
#define MAX_PAWS (4)

/*****
Typedef definitions
*****/
typedef uint16_t doggy_t;
typedef struct
{
    uint8_t paws[MAX_PAWS];
    uint8_t tail;
    uint8_t nose;
} neko_t;

/*****
Exported global variables
*****/
extern uint8_t g_example_Distance;
extern const char g_example_ReplaceText[];

/*****
Exported global functions (to be accessed by other files)
*****/
uint8_t R_EXAMPLE_Foo(uint16_t address, uint8_t size);
uint8_t R_EXAMPLE_Bar(uint8_t num_paws);

```

Rationale:

Grouping these sections at the top of source and header files makes it easier for users to identify and reference this information.

3.1.3. Style_Format_003

All file headers shall have a revision history which includes correct information about:

1. Version.
2. Date of release/publish.
3. Change description.

* Note: Revision history should contain appropriate change description whenever a change request is implemented.

Example:

Not required.

Rationale:

To record the changes of a file during project.

3.1.4. Style_Format_004

All function headers shall have a information about:

1. Function name.
2. Purpose.
3. Parameters.
4. Return value.

Example:

Not required.

Rationale:

Clarification.

3.1.5. Style_Format_005

Each line in source files shall be limited to 120 columns (more strict rule can be defined as project specific). If the statement exceeds 120 characters, use a 'carriage return' and indent 2 spaces to continue the statement on the next line.

Example:

Not required.

Rationale:

Readability and compatibility

3.1.6. Style_Format_006

Tabs shall not be used instead spaces shall be used. Maintain 2 spaces difference between the levels of indentation.

Example:

Not required.

Rationale:

Different text editors/printers expand tabs to different number of spaces which modifies the original layout of the file.

3.1.7. Style_Format_007

A space shall be used between *'if'*, *'else if'*, *'while'*, *'for'*, *'switch'* and the opening parenthesis.

Example:

```
/* Compliant */  
if (Spi_Var > 10)  
while (Spi_Var != 0)
```

```
/* Not compliant */  
if (Spi_Var>10)  
while (Spi_Var!=0)
```

Rationale:

Readability

3.1.8. Style_Format_008

Spaces shall not be used around *'.'*, *'->'* operators.

Spaces shall not be used between unary operators and operands.

Unary operators: *'++'*, *'--'*, *'~'*, *'!'* *'&'* (address of), *'*'* (pointer).

Example:

```
/* Compliant */  
Spi_StaticCfg.Addr  
Spi_StaticCfg->Addr  
Spi_RxCntr++;  
&Spi_RxBuffer
```

```
/* Not compliant */  
Spi_StaticCfg . Addr  
Spi_StaticCfg -> Addr  
Spi_RxCntr ++;  
& Spi_RxBuffer
```

Rationale:

Methods to prevent implausible values, execution errors, and errors in the data flow and control flow.

3.1.9. Style_Format_009

A space shall be used before and after a binary operator.

Example:

```
/* Compliant */  
Sum = Data + Data2;
```

```
/* Not compliant */  
Sum=Data+Data2;
```

Rationale:
Readability

3.1.10. Style_Format_010

A space shall be used after each comma in the argument or initialization list.

Example:

```
/* Compliant */  
Sum = Utl_Sum(Num1, Num2, Num3);
```

```
/* Not compliant */  
Sum = Utl_Sum(Num1,Num2,Num3);
```

Rationale:
Readability

3.1.11. Style_Format_011

Each expression statement shall be on a separate line.

Example:

```
/* Not compliant */  
/* It has two statements in one line */  
uc_Sum = uc_num1 + uc_num2; uc_Avg = uc_Sum / 2;
```

```
/* Compliant */  
/* It has two statements in two lines */  
uc_Sum = uc_num1 + uc_num2;  
uc_Avg = uc_Sum / 2;
```

Rationale:
Simplicity and readability

3.1.12. Style_Format_012

Braces which enclose a block shall be placed in the same column, on separate lines directly before and after the block. There shall not be anything else on these lines. The block of code inside the opening and closing braces shall be indented 2 spaces. An opening brace is always at the same level as the line above.

Exception:

1. The *'while'* in the *'do...while'* statement may be placed one space character after the brace closing the block.
2. The name of the new user type (*'struct'*/*'union'*/*'enum'* type) may be placed one space after the brace closing the element definitions.
3. Array initialization where the opening and closing braces in same line.

Example:**/* Compliant */****a)**

```

if (TRUE == RAM_CopyData)
{
    /* if block */
}
else
{
    /* else block */
}

```

b)

```

void Eep_Cancel(void)
{
    /* Function block */
}

```

c)

```

do
{
    /* do-while block */
} while (Luc_Cntr != 0)

/* Not compliant */
a)
if (TRUE == RAM_CopyData)
{
    /* if block */
}
else
{
    /* else block */
}

```

b)

```

void Eep_Cancel(void) {
}

```

Rationale:

Simplicity and readability

3.1.13. Style_Format_013

'*if*' statements ('*#if*' - '*#else*' - '*#endif*') should be indented according to their nesting depth.

Example:**/* Compliant */**

```

if (TRUE == RAM_CopyData)
{
    #if ( )
    /* #if block */
    #endif
}

```

```

/* Not compliant */
if (TRUE == RAM_CopyData)
{
    #if ( )
    /* #if block */
    #endif
}

```

Rationale:

Simplicity and readability

3.1.14. Style_Format_014

Tabs and unwanted spaces shall not be present at the end of the line in make files.

Example:

At the end of line, need remove the space, and tab.

Rationale:

To avoid compilation error because of tabs and unwanted spaces at the end of the line in make files.

3.1.15. Style_Format_015

The constants shall be place on the left of equality comparisons.

Example:

```

#define CONSTANT 10
uint8 LucVariable = 0;

```

/* Compliant */

```

if (CONSTANT == LucVariable)
{
    /* */
}

```

/* Not compliant */

```

if (LucVariable == CONSTANT)
{
    /* */
}

```

Rationale:

Prefer the use of “*if (CONSTANT == LucVariable)*” over “*if (LucVariable == CONSTANT)*” in order to enable the automatic check of a quite common error of “*if (LucVariable = CONSTANT)*”. The latter represents unwanted assignment and produces unexpected results.

3.1.16. Style_Format_016

Check that all condition checks have variables of same type on RHS and LHS or there is explicit type casting.

RHS: right-hand side

LHS: left-hand side

Example:

```

/* Compliant */
#define CONSTANT (uint8) 10
uint8 LucVariable = 0;
if (CONSTANT == LucVariable)
{
    /* */
}

```

Rationale:

To avoid illegal access of variables and avoid compile warning.

3.1.17. Style_Format_017

Rules for the assembly language using, should be applied defined as a project-specific coding rules

Example:

Not required.

Rationale:

Not required.

3.2. File Structure**3.2.1. Style_File_001**

The interface file shall have the extension '.h' and the body file shall have the extension '.c'.

Example:

Not required

Rationale:

Apply praxis

3.2.2. Style_File_002

Code duplication shall be avoided. The use of functions shall be implemented to increase maintainability and portability of source code.

* Note: Functions that are common shall be placed in a separate file.

Example:

The write-verify safety mechanisms are using everytime write value to register. It better to define the function to handle it.

```

/* Not Compliant */
/* Write verify for register RSCFDnTMIEC0 */
/* Save the written value to compare */
LucCompareValue = CAN_ENABLE_MESSAGE_BUFFER_INT;
/* Enable message buffer interrupt */
(*LpRSCFDnTMIEC0Reg) = CAN_ENABLE_MESSAGE_BUFFER_INT;

```

```

/* Read value from register RSCFDnTMIEC0 */
LucCheckValue = (*LpRSCFDnTMIEC0Reg)
if (LucCompareValue == LucCheckValue)
{
    LucResult = E_OK;
}

/* Compliant */
/* Write verify function will return E_OK if the value is written completed
*/
LucResult = CAN_WriteVerify(LpRSCFDnTMIEC0Reg,
CAN_ENABLE_MESSAGE_BUFFER_INT);

```

Rationale:

- To avoid memory wastage.
- Improve portability and maintainability.
- Readability and understandability.

3.2.3. Style_File_003

The file (both header and source) should end with a blank line.

Example:

Not required

Rationale:

If the file is inserted in another file using ‘*#include*’, it may lead to errors if the last line of the included file is a comment.

3.2.4. Style_File_004

If a module provides several functions and processes additional module source files “<Module>_<Sub>.c” shall be used. Meaningful Name can be chosen freely for “<Sub>”.

Example:

Not required

Rationale:

Sub module c files can help to structure a module.

A linker locates only complete objects derived from a file and such an object cannot be subdivided.

If more sub files exists linker can work more efficient.

3.2.5. Style_File_005

The header files should not be included back to back. One header file included in another header file which is included back in the first header file.

Example:

Not required

Rationale:

Not required

3.2.6. Style_File_006

A ".c" file shall not be included in another file: it shall be compiled and provided as an object module.

Example:

```
/* Not compliant */  
#include "Can.c"
```

Rationale:

Reduce include effort

3.3. Comments

3.3.1. Style_Comment_001

Multi-line comment blocks shall have the opening and closing character sequences in separate lines. There shall not be anything else on these lines, but optional star frames.

Example:

```
/*  
 * This is a multi-line comment block line 1  
 * This is a multi-line comment block line 2  
 */  
  
/*****  
 * This is a multi-line comment block with a star frame *  
 * This is a multi-line comment block with a star frame *  
 *****/
```

Rationale:

Readability and uniformity

3.3.2. Style_Comment_002

Comments shall comply with the ANSI-C Standard.
C++ comments are not permitted.

Example:

```
/* Compliant */  
/* Comment */  
  
/* Not compliant */  
// Comment
```

Rationale:

Readability

3.3.3. Style_Comment_003

For the following elements, suitable comments shall be added:

- Includes (for what is it needed?).
- Definition of macros.
- Declaration of types.
- Elements of structures and unions.
- Definition of variables.
- Definition of constants.
- Definition of functions.

Example:

Not required

Rationale:

Uniformity and readability

3.3.4. Style_Comment_004

The following C constructs shall be commented after the concluding bracket if they are longer than 10 lines:

'switch-case'

'if-then-else'

'do-while'

'for'

Example:

```
if (a < b)
{
    /*
        More than 10 lines
    */
} /* end of if (a < b) */
```

Rationale:

Readability

3.3.5. Style_Comment_005

Constructs and compiler switches shall be commented at the end of block (e.g. after the concluding bracket, *'#endif'*, etc) if they are longer than 10 lines.

Example:

```
1)
/* Compliant */
if (a < b)
{
    /*
        More than 10 lines
    */
} /* end of if (a < b) */
```

2)


```

/* Compliant */
#if (CONDITION == TRUE)
/*
    more than 10 lines
*/
#endif /* (CONDITION == TRUE) */

```

Rationale:

Readability

3.3.6. Style_Comment_006

Comments shall not be nested.

Example:

```

/* Not compliant */
/* Comment /* this is a Nested comment */ //another nested comment */

```

Rationale:

Nested comments may create problems with some compilers.

3.3.7. Style_Comment_007

A comment should be placed immediately above and indented same as the line/block of code which it is complementary.

Example:

```

# Compliant
/* Check CONDITION */
if (CONDITION == FALSE)
{
    /* Change CONDITION */
    CONDITION = TRUE;
    .....
}

```

Rationale:

Readability and understandability.

4. Coding Rules

4.1. Conformance to external rules

4.1.1. Rules_Ext_001

If a MISRA warning has occurred, analyse the warning to know if that error can be avoided by modifications in the code. Or that violation of MISRA rule shall be commented and reasoned at the corresponding code line.

Example:

Not required

Rationale:

Not required

4.1.2. Rules_Ext_002

Conform to the atomic data specification defined by the MCU architecture unless there is a particular request.

* Note: On 32-bit CPU architectures access to naturally aligned 8-bit, 16-bit and 32-bit values is atomic. As a consequence no lock for direct simple accesses is needed (but read-modify-write instructions, e.g. ++, --, +=, -=, etc. are not atomic and still have to be protected).

Example:

Not required

Rationale:

Not required

4.2. Environment

4.2.1. Rules_Envr_001

The software modules have to be developed to be compliant for all processor platforms without any changes. Any necessary processor or compiler specific instructions (e.g. memory locators, pragmas, use of atomic bit manipulations etc.) have to be exported to macros and include files.

Example:

Not required

Rationale:

To avoid major rework due to change in compiler and processor specific changes.

4.2.2. Rules_Envr_002

All SOFTWARE Modules shall not use compiler or platform specific keywords directly.

Example:

If specific keywords are needed, they shall be redefined (mapped) as follows:

'Compiler.h':

```
#define FAR(X)    __far__ (X);
```

Usage of macro within source code:

```
FAR(void) function();
```

Rationale:

To avoid major rework due to change in compiler and processor specific changes.

4.2.3. Rules_Envr_003

While using object/library files (from third party, customer, etc.) it shall be confirmed that it was produced with same compilers and version which is used in the project.

Example:

Not required

Rationale:

Correct integration

4.2.4. Rules_Envr_004

There should not be any compiler and linker warnings (of-course errors also) while building the software.

Example:

Not required

Rationale:

Warnings are meant to be corrected.

4.2.5. Rules_Envr_005

ANSI-C does not support inline assembler and there is no general keyword for the compilers (GHS, Renesas compiler). Therefore a macro shall be added in the compiler abstraction to simplify porting to different compilers.

Example:

```
/* Not compliant */  
__asm("hault");
```

```
/* Compliant */  
ASM_HAULT();
```

Rationale:

To have a compiler independent code.

4.2.6. Rules_Envr_006

For runtime software, Compiler specific header files, libraries and intrinsic functions shall not be used.

Example:

Not required

Rationale:

Portability and understandability.

4.3. Comments/Documentation

4.3.1. Rules_Comment_001

All comments shall be in English and spell checked.

Example:

```
/* Compliant */
/* Set Interrupt Source Flag check */
Lb1CANIrqSrcFlag = CAN_TRUE;

/* Not compliant */
/* 割り込みソースフラグチェックを設定 */
Lb1CANIrqSrcFlag = CAN_TRUE;
```

Rationale:

Readability

4.3.2. Rules_Comment_002

The purpose of every line of executable code should be explained by a comment, although one comment may describe more than one source line of code.

Example:

Not required

Rationale:

Readability

4.3.3. Rules_Comment_003

Comment should be meaningful with the corresponding line of code. Avoid to stating in comments what is better stated in code.

Example:

```
/* Compliant */
/* If message buffer interrupt configuration is set as ON, enable it */
if (STD_ON == CAN_MESSAGE_BUFFER_INT)
{
    (*LpRSCFDnTMIEC0Reg) = CAN_ENABLE_MESSAGE_BUFFER_INT;
```

```

}

/* Not Compliant */
/* Check CAN_MESSAGE_BUFFER_INT is equal with STD_ON or not */
if (STD_ON == CAN_MESSAGE_BUFFER_INT)
{
/* Write CAN_ENABLE_MESSAGE_BUFFER_INT to RSCFDnTMIEC0 register */
(*LpRSCFDnTMIEC0Reg) = CAN_ENABLE_MESSAGE_BUFFER_INT;
}

```

Rationale:

Redundant comments are unnecessary and increase the maintenance effort.

4.3.4. Rules_Comment_004

Commented out code shall not be left permanently in source files. Any lines or blocks of code that exists specifically for debugging purpose shall be surrounded by pre-processor directives for conditional compilation (e.g. `#ifdef DEBUG ... #endif`).

Example:

Not required

Rationale:

Redundant comments are unnecessary and increase the maintenance effort.

4.3.5. Rules_Comment_005

File/function header format should be consistent across the code.

Example:

```

/*****
* Function name: MyFunc(uint8 X)
* Description: Short Description, including preconditions
* Parameter X: Description
* Return value: None
* Remarks: global variables used, side effects
*****/
void MyFunc(uint8 X)
{
}

```

Rationale:

Readability and understandability.

4.3.6. Rules_Comment_006

Each cast shall feature an associated comment describing how the code ensures proper behaviour across the range of possible values on the right side.

Example:

```

/* Compliant */
/* Function prototype */
uint8 R_EXAMPLE_FooBar(uint8 arg);

```

```
uint16 num;
uint8 result;

/* Only the first byte of variable 'num' is necessary */
result = R_EXAMPLE_FooBar((uint8)num);
```

Rationale:

Readability and understandability

4.4. Identifiers

4.4.1. Rules_Identifier_001

Identifiers must not simultaneously have both internal and external linkage in the same translation unit.

Example:

```
static uint8 Adc_Data1 = 0u;

void Adc_FunctionX(void)
{
    /* Not compliant */
    extern uint8 Adc_Data1;

    /* ... */
}
```

Rationale:

Avoid variable name hiding which can be confusing.

4.5. Types

4.5.1. Rules_Types_001

Enumeration types must be used instead of integer types and constants to select from a limited series of choices.

Example:

```
/* Compliant */
typedef enum ETag_Can_HwFaultStatus
{
    CAN_HW_NO_FAULT,
    CAN_HW_SHORT_TO_GND,
    CAN_HW_SHORT_TO_VCC,
} Can_HwFaultStatus;

Can_HwFaultStatus Can_FaultStatus;

/* Not compliant */
#define CAN_HW_NO_FAULT (uint8)0
#define CAN_HW_SHORT_TO_GND (uint8)1
#define CAN_HW_SHORT_TO_VCC (uint8)2
```

```
uint8 Can_HwFaultStatus;
```

Rationale:

Enhances debugging, readability and maintenance.

Compiler or static check tool can be set to generate a warning when the ‘enum’ type variable is used in ‘switch’ statement and all enumerators are not used as case.

4.5.2. Rules_Types_002

Integer values of the enumeration elements must not be used in calculations.

Example:

```
typedef enum ETag_Element
{
    ELEMENT_1,
    ELEMENT_2,
    ELEMENT_3
} Element;
```

```
Element Variable;
```

```
/* Not compliant */
```

```
Variable = ELEMENT_1 + 3;
```

Rationale:

The integer value of an enumeration element can be changed when another element is added at later point of time during development.

4.5.3. Rules_Types_003

Identifiers shall not contain the ‘_’ character twice in succession.

Example:

```
/* Compliant */
```

```
Com_GucConfigRefType
```

```
/* Not compliant */
```

```
Com__GucConfigRefType
```

Rationale:

Readability, allow system services.

4.5.4. Rules_Types_004

Variables for loop counters must be declared in the generic type (e.g. ‘uint’).

Example:

Not required

Rationale:

The variables declared in generic type could be handled best by all of the compilers. Therefore loop counters, in view of reuse of software, have to be declared in the generic type.

4.6. Constants

4.6.1. Rules_Const_001

Hexadecimal constants shall be represented using all uppercase letters following 0x.

Example:

```
#define RAM_START_ADDR    0x7AFEu /* Compliant */  
#define RAM_START_ADDR    0x7afeu /* Not compliant */
```

Rationale:

Readability

4.6.2. Rules_Const_002

Numeric values shall not be used in code; symbolic values shall be used instead.

Exception:

1. 0 in initialization of variables.
2. 1 in bit operations.
3. Constants used in generic math expressions like 2 in calculating the average value of two numbers.

Example:

Not required

Rationale:

Readability and maintenance

4.6.3. Rules_Const_003

All SOFTWARE Modules shall indicate all global data with read-only purposes by explicitly assigning the 'const' keyword.

Example:

```
uint8 const MaxPayload = 0x18u;
```

Rationale:

The variable shall be stored in ROM and should not be changeable during runtime.

4.7. Declarations and Definitions

4.7.1. Rules_Defn_Decl_001

Global variables must be avoided or its usage needs to be justified.

Example:

Not required

Rationale:

The module which uses the global variable can update mistakenly and it is difficult to debug.

4.7.2. Rules_Defn_Decl_002

The *'register'* storage class specifier must not be used.

Example:

```
/* Not compliant */  
register uint8 Adc_Data = 0u;
```

```
/* Compliant */  
static uint8 Adc_Data = 0u;
```

Rationale:

Compiler technology is now capable of optimal register placement and some compilers ignore the *'register'* storage specifier.

4.7.3. Rules_Defn_Decl_003

A structure or enumeration type variable shall not be defined in the type definition itself. It means each self-defined type has to have an explicit type declaration even if there is only one variable of this type.

Example:

```
/* Not compliant */  
enum  
{  
    DIR_RIGHT,  
    DIR_LEFT  
} Ctl_DriverSide;  
  
/* Compliant */  
typedef enum ETag_Ctl_DriverSideType  
{  
    DIR_RIGHT,  
    DIR_LEFT  
} Ctl_DriverSideType;  
  
Ctl_DriverSideType Ctl_DriverSide;
```

Rationale:

Readability

4.7.4. Rules_Defn_Decl_004

Multiple variable declarations shall not be allowed on the same line.

Example:

```
/* Compliant */
```

```
uint8 Can_RxData;
uint8 Can_TxData;

/* Not compliant */
uint8 Can_RxData, Can_TxData;
```

Rationale:
Readability

4.7.5. Rules_Defn_Decl_005

A global/exported object or function of a module shall be declared with ‘extern’ only in that particular module header file.

Example:

Using the CAN module object/function in “Apl_Process.c” like below is not compliant

```
/* Not compliant */
extern uint8 Can_RxData;

void Apl_ProcessCan(void)
{
    if (0 == Can_RxData)
    {
        /* do some action */
    }
}

/* Compliant */
#include "Can_Receive.h"

void Apl_ProcessCan(void)
{
    if (0 == Can_RxData)
    {
        /* do some action */
    }
}
```

Rationale:

Allow access to global functions and variables by including the header file of the defining module.

4.7.6. Rules_Defn_Decl_006

Each implementation file shall have the corresponding header file and an optional private header file.

Example:

Not required

Rationale:

Private functions and variables shall not be sharable to other modules.

4.7.7. Rules_Defn_Decl_007

A private header file must not be used by implementation files other than the one it corresponds to.

Example:

Not required

Rationale:

To make the module atomic and avoid interference between modules.

4.7.8. Rules_Defn_Decl_008

Variables and functions (other than macros) shall not be defined within in an '.h' file. They shall be defined within the module's C file.

Example:

In 'lcu.h', we declare "uint8 lcu_IntCounter". Then in 'lcu.c' and 'lcu_Cfg.c', both of them include 'lcu.h'. So when compilling, there will be error. To avoid, we shall declare "uint8 lcu_IntCounter" in 'lcu.c' and 'extern' that variable in 'lcu.h' or 'lcu_Cfg.c'.

Rationale:

Prevent multiple declarations (i.e. linker problem).

4.7.9. Rules_Defn_Decl_009

Directly use compiler and platform specific inline keywords like '`__inline__`' or '`_inline`' are not allowed. To define an inline function, macro (e.g. `LOCAL_INLINE`) shall be used.

*** Note:**

- Check if there are any inline public functions. If so, make a note. This is necessary since the compiler option has to be changed for such files.
- Prototype and definition has to be placed in a header file (if the inline function is used multiple times in different c files) or it can be placed in the c file (if the inline function is only called inside the c file).

Example:

```
/* Compliant */
/* Prototype */
LOCAL_INLINE uint8 MyModule_Func(sint16 arg1, sint16 arg2);

/* Definition */
LOCAL_INLINE uint8 MyModule_Func(sint16 arg1, sint16 arg2)
{
    /* Code */
}

/* The inline function can be called within a c file */
/* Call of the inline function */
MyModule_Func(MyArg1, MyArg2);
```

Rationale:

To encapsulate all needed keywords and properties to define an inline function.

4.7.10. Rules_Defn_Decl_010

Declarations of functions shall always be stated with detailed parameter list, i.e. the type and a practical designation of the relevant parameters.
Designator names in '.c' and '.h' file shall be identical.

Example:

```
extern Std_ReturnType Eep_Erase
(
    Eep_AddressType EepromAddress,
    Eep_LengthType Length
);
```

Rationale:

Readability and correct use of API.

4.7.11. Rules_Defn_Decl_011

Declaration and definition of local functions shall have the storage-class specifier '*static*'.
Local function means function with internal linkage (only visible inside the module).

Example:

```
static void MyLocalFunction(void);
```

Rationale:

Limit visibility of local functions.

4.7.12. Rules_Defn_Decl_012

Variables used in the code need to be defined; moreover, unused variables should not be contained.

Example:

Not required

Rationale:

In order to prevent redundancy and compiler warnings.

4.7.13. Rules_Defn_Decl_013

Unused input parameter(s) of function shall not be declared.

Example:

Not required

Rationale:

To avoid redundancy and compiler warnings.

4.7.14. Rules_Defn_Decl_014

The length of the register in the device header file and the CPU manual should be verified and its implementation and the value generation details should be verified across variants to avoid illegal access of that register.

Example:

If the register in use is 8 bits but due to mistake in implementation the access of that register is 32 bits then this will be considered as illegal access.

Rationale:

To avoid illegal access.

4.7.15. Rules_Defn_Decl_015

The ‘volatile’ keyword should be used to indicate variables that may be modified outside of the program’s control (i.e., any register values that are set or cleared by hardware, or memory that is dual-port and may be modified by an external device).

Example:

Not required

Rationale:

If this is not done, the optimizer may make optimizations that will result in incorrect operation.

4.7.16. Rules_Defn_Decl_016

The common implementations across the modules such as initialization of status of the driver should be consistent across all the modules.

Example:

```
/* Defined values for compile switch conditions in all modules are "STD_ON" and "STD_OFF". */
/* Compliant */
/* Module DIO */
#if (DIO_CHANNEL_CONFIGURED == STD_ON)
#if (DIO_CHANNELGROUP_CONFIGURED == STD_ON)

/* Module MCU */
#if (MCU_DEV_ERROR_DETECT == STD_ON)
#if (MCU_NO_PLL == STD_OFF)
```

Rationale:

Uniformity and easy understanding.

4.7.17. Rules_Defn_Decl_017

The variables and macros (used in different modules but similar context), shall be named similarly. Consistency should be maintained across modules.

Example:

Name of the status variable can be consistent across all modules, e.g.
- ADC module: ‘Adc_GblDriverStatus’.

- GPT module: 'Gpt_GblDriverStatus'.
- PWM module: 'Pwm_GblDriverStatus'.

Rationale:

Uniformity and easy understanding.

4.7.18. Rules_Defn_Decl_018

The type of variable should be appropriately selected depending on the values that the variable may hold.

Example:

For example, if the variable can hold 'INIT', 'UNINIT' and 'BUSY', then enum or integer needs to be given as type instead of boolean.

Rationale:

To avoid unexpected behaviour.

4.7.19. Rules_Defn_Decl_019

Typecasts and the values assigned to variables, the compiler abstractions to it shall be correctly defined and shall be consistent across the modules.

Example:

Not required

Rationale:

Uniformity

4.7.20. Rules_Defn_Decl_020

No multiple uses of variable names.

Example:

```
/* Not compliant */
/* Using global variable and local variable with the same name */
unsigned int returnValue;

unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    unsigned int returnValue = 0;
    if(LuiVal1 > LuiVal2)
    {
        returnValue = LuiVal1;
    }
    else
    {
        returnValue = LuiVal2;
    }
    return returnValue;
}
```

```

/* Compliant */
/* No using global variable and local variable with the same name */
unsigned int Adc_GuiReturnValue;

unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    unsigned int LuiReturnValue = 0;
    if(LuiVal1 > LuiVal2)
    {
        LuiReturnValue = LuiVal1;
    }
    else
    {
        LuiReturnValue = LuiVal2;
    }
    return LuiReturnValue;
}

```

Rationale:

To avoid errors in implementation.

4.7.21. Rules_Defn_Decl_021

Bit-fields shall not be defined within signed integer types.

Example:

Not required

Rationale:

Several details of the manipulation of binary data within signed integer containers are implementation-defined behaviours of the C standard.

4.7.22. Rules_Defn_Decl_022

If a function parameter is not going to be changed inside the function then it should be made a constant parameter.

Example:

```
void func(const int nFirstNumber, const int nSecondNumber)
```

Rationale:

Readability

4.7.23. Rules_Defn_Decl_023

The keyword '*auto*' shall not be used ('*auto*' is the default storage class for local variables).

Example:

Not required.

Rationale:

The '*auto*' keyword is an unnecessary historical feature of the language.

4.7.24. Rules_Defn_Decl_024

Variables shall be defined in the appropriate scope. for example, the variable that is defined in the same file and is accessed from multiple functions, thie variables declared as static at file scope.

Example:

```
/* Not Compliant */
/* The Guc_Variable is declared as global variable but just used in
functionA() */
uint8 Guc_Variable = 1;
```

```
uint8 functionA()
{
    return Guc_Variable;
}
```

```
uint8 functionB()
{
/* functionB don't use Guc_Variable */
}
```

```
/* Compliant */
/* The Luc_Variable is used within functionA, so it should be declared as
local variable. */
```

```
uint8 functionA()
{
    uint8 Luc_Variable = 1;
    return Luc_Variable;
}

uint8 functionB()
{
/* functionB don't use Luc_Variable */
}
```

Rationale:

Performance and to avoid illegal access.

4.7.25. Rules_Defn_Decl_025

the variables should have only one purpose. should not have multiple purpose.

Example:

Not required.

Rationale:

Readability and to avoid illegal access.

4.7.26. Rules_Defn_Decl_026

The Type and size of the variables are suited to its intended use.

e.g. Do not increase the occupied size by taking on extra the size of the variable.

Example:

```
/* Not compliant */
unsigned int Adc_GuiReturnValue;

unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    GuiReturnValue = 0x04U;
}
```

```
/* Compliant */
unsigned char Adc_GuiReturnValue;

unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    GuiReturnValue = 0x04U;
}
```

Rationale:

Readability and to avoid illegal access.

4.8. Initialization

4.8.1. Rules_Init_001

All variables and pointers shall be initialized to its default value before its usage. The declaration and initialization shall be in different lines.

Example:

```
/* Compliant */
uint8 LucCount;
LucCount = CAN_ZERO;
```

```
/* Not compliant */
uint8 LucCount = CAN_ZERO;
```

Rationale:

To avoid initialisation of variables with garbage values and for readability.

4.9. Control Statement Expressions

4.9.1. Rules_Expr_001

Brackets shall always be used in complex expressions even if the C priority rules do not necessarily demand this for operators. This also applies to expressions evaluated by the pre-processor.

Example:

```
/* Compliant */  
if ((counter1 > 0) || (counter2 < 0))  
#if ((SEC_SECURITY == C) || (SEC_SECURITY_CLASS == CCC))
```

Rationale:

Readability

4.9.2. Rules_Expr_002

Control structure nesting should be minimized and not exceed 4 levels. The number of levels shall be customizable. That means that it shall be possible to reduce the allowed number of levels per file.

Example:

Not required

Rationale:

Less complex and easy understanding.

4.9.3. Rules_Expr_003

Multiple assignments shall not be done.

Example:

```
/* Local/static variable of type 'sint16' */  
static sint16 Adc_VarX;  
static sint16 Adc_VarY;  
static sint16 Adc_VarZ;
```

/* Compliant */

```
Adc_VarX = Adc_VarY;  
Adc_VarY = Adc_VarZ;
```

/* Not compliant */

```
Adc_VarX = Adc_VarY = Adc_VarZ;
```

Rationale:

Readability

4.9.4. Rules_Expr_004

The use of '++' and '--' should be limited to simple cases. They shall not be used in statements where other operators occur. The prefix use is always forbidden.

Example:

```
/* Local/static variable of type 'uint16' */  
static uint16 LusVarX;  
static uint16 Adc_GusVarY;  
  
/* Not compliant */  
uint16 LusVarX -= Adc_GusVarY++;
```

Rationale:

Readability and maintenance

4.9.5. Rules_Expr_005

While performing masking operation, make sure of the mask value to be used and its impact for other functionalities.

Example:

Not required

Rationale:

To avoid bugs in implementation.

4.9.6. Rules_Expr_006

Check for wrong usage of "&&", "||" while performing mask operations in the CODE.

Example:

```
1 && 2 = true  
because  
1 = true (non-zero) in C  
2 = true (non-zero) in C  
But  
1 & 2 = 0 = false  
because  
1 = 0001 in binary  
2 = 0010 in binary
```

Rationale:

To avoid bugs in implementation.

4.9.7. Rules_Expr_007

If the division operation is being performed in the embedded code then make sure to avoid condition of illegal division by zero.

Example:

```
/* Compliant */
```

```

if (c != 0)
{
    a = b / c;
}

```

Rationale:

To avoid hanging of code while execution.

4.9.8. Rules_Expr_008

During updating for similar part of code (e.g. timer unit/channel interrupt handlers), check the code (e.g. unit/channel numbers) carefully, because there can be typo mistakes in updating correct values of those numbers.

Example:

Not required

Rationale:

To avoid bug in implementation.

4.9.9. Rules_Expr_009

Try to avoid comparison of RAM variable in the loops to reduce the CPU load.

Example:

```

(LucLoop < (uint8_least)LpRunTimeData->ucChannelCount)

```

In the above example, the comparison is made exactly the same number of time as there are channels configured. But RAM access is slower than register access, so use a local variable that should hold the channel count. Channel count should be read once before loop execution (a single memory access). Using a local variable the compiler will assign an internal register to the variable. This way the 'for' loop comparison operator will be translated as a register to register comparison - avoiding undesired and slow RAM access.

Rationale:

To reduce the execution time.

4.9.10. Rules_Expr_010

Considering integer boundaries (overflows & underflows) while handling arithmetic operation.

Example:

- Overflow of unsigned integers:
32 bits integer types can hold certain ranges of values.

So if we have two unsigned integer types each with the value of 2147483648 (a & b):

$a + b = 4294967296$

which is larger than the maximum value that can be represented in an unsigned integer type. This is called an integer overflow.

- Underflow of unsigned integers:
unsigned int a, b;

```
a = 0
b = a - 1
```

The value of b is -1, which is below than the minimum possible value that can be stored this is called an integer underflow.

- Overflow/Underflow of Signed Integers:

The signed integer two's complement representation in binary will have value, padding, & sign bits. The sign bit represents the sign of the integer 0 for positive and 1 if the number is negative. When an overflow or underflow condition occurs on signed integers the result will wrap around the sign and causes a change in sign.

For example:

A 32 bit number 2147483647 = 0x7FFFFFFF in hex.

If we add 1 to this number it will be 0x80000000 which is equivalent to -2147483648 decimal.

With signed addition or subtraction, you can overflow the sign boundary by causing a positive number to wrap around 0x80000000 and become a negative number. You can also underflow the sign boundary by causing a negative number to wrap below 0x80000000 and become a positive number.

Note: When result of arithmetic operation is stored in HW registers, the maximum value which register can hold to be considered before loading result of arithmetic operation into a register. If result value exceeds the limit which register can hold, work around to be carried out before loading result value into a register.

Rational:

A numeric overflow or underflow that occurs early in a block of code can lead to a subtle series of cascading faults; not only is the result of a single arithmetic operation tainted, but every subsequent operation using that tainted result introduces a point where an attacker might have unexpected influence.

4.9.11. Rules_Expr_011

No dynamic objects or variables, or else online test during their creation.

Example:

Don't use function '*malloc()*' to allocate memory in program.

Rationale:

It's helpful for easily control memory.

4.9.12. Rules_Expr_012

Type conversions of the variables shall be done explicitly and correctly. It shall not be implicit.

Example:

```
1)
extern void foo(uint8 ucVal);
uint8 ucValue;
sint16 ssValue;
sint32 slValue;
void func(void)
```

```
{
/* Compliant */
foo(ucValue);

/* Not compliant */
foo(ssValue);
foo(slValue);
}
```

```
2)
/* Compliant */
(uint16) 0xFFFF
(uint32) 0x10000

/* Not compliant */
(uint16) 0x10000
```

Rationale:

Implicit type conversion may lead to data loss and incorrect type casting of variables may cause bugs in implementation.

4.9.13. Rules_Expr_013

None of the bit-wise operators (i.e. '&', '|', '~', '^', '<<', '>>') shall be used to manipulate signed integer data.

Example:

Not required

Rationale:

Several details of the manipulation of binary data within signed integer containers are implementation-defined behaviours of the C standard.

4.10. Control Flow

4.10.1. Rules_CtrlFlow_001

The controlling expressions shall not contain function call. The following list is examples of the controlling expressions:

1. 'if' statement.
2. 'for' statement.
3. 'while' statement.
4. 'do-while' statement.
5. 'switch' statement.

Example:

```
/* Not compliant */
while (Mod_GetIndex() > 42u)
```

```
/* Not compliant */
for (Luc_Index = Mod_GetIndex(); Luc_Index < 10u; Luc_Index++)
```

Rationale:

Function may not get executed in some cases, if intended to execute.

4.10.2. Rules_CtrlFlow_002

Assignments shall not be made within an *'if'* or *'else if'* or loop controlling expression (except for a single loop counter initialization in the first clause of a *'for'* statement).

Example:

1)

```
/* Compliant */
```

```
ret = R_EXAMPLE_FooBar(arg1);
```

```
if (ZERO == ret)
```

```
/* Not compliant */
```

```
if (ZERO == (ret = R_EXAMPLE_FooBar(arg1)))
```

2)

```
/* Compliant */
```

```
/* loop counter initialization for 'for' statement */
```

```
for (counter = 0; counter < MAX_COUNT; counter++)
```

Rationale:

To avoid unexpected control flows.

4.10.3. Rules_CtrlFlow_003

Cyclomatic complexity numbers of all functions shall be minimized and be limited for each project (basically ≤ 10 for each ASIL level is reasonable). They shall be analysed and documented. The limitation should be set less than 30 dependent on the specific project.

Example:

Not required

Rationale:

Too complex functions are difficult to read, test and maintain.

4.10.4. Rules_CtrlFlow_004

'break' and *'continue'* commands should never be used in loops except under very rare circumstances when there is no other option.

Example:

```
/* Compliant */
```

```
while (nX < 10)
```

```
{
```

```
    nX++;
```

```
}
```

```
/* Not compliant */
```

```
while (TRUE)
```

```
{
```

```

    nX++;
    if (nX == 10)
    {
        break;
    }
}

```

Rationale:

- To avoid endless loops.
- To avoid bugs in implementation.

4.10.5. Rules_CtrlFlow_005

Any 'if' statement shall end with an 'else' clause.

Example:

```

/* Not compliant */
if (MOD_GetIndex() > 42u)

/* Compliant */
if (MOD_GetIndex() > 42u)
{
}
else
{
    /* No action required */
}

```

Rationale:

Understandably

4.10.6. Rules_CtrlFlow_006

Avoid endless loop conditions in the source code (especially the loops in source code which have exit conditions depending on changes in the status of registers). Any loop should also have a condition to exit at some points of time by using some count values which are appropriate for the functionality of that loop.

Example:

```

/* Compliant */
while ((FALSE == LucExitFlag) && (ZERO < LucTimeOut))
{
    /* ... */
    LucExitFlag = REGISTER_STATUS;
    LucTimeOut--;
}

/* Not compliant */
while (FALSE == LucExitFlag)
{
    /* ... */
    LucExitFlag = REGISTER_STATUS;
}

```


Rationale:

The code may hang if the loop condition does not satisfy.

4.10.7. Rules_CtrlFlow_007

The '*break*' should be between each '*case*' statement, unless the '*case*' statements need to be executed in sequence in switch statements (mentioned exception is allowed with regard to faster processing speed, higher code efficiency, etc.).

* Note: All switch statements shall have the default case implemented, even if it is left empty apart from the break statement.

Example:

Not required

Rationale:

Unnecessary code execution may lead to wrong functionality.

4.10.8. Rules_CtrlFlow_008

The write protected registers should be identified.

The special sequence of using protection command registers and status registers to update those protected registers should be followed.

Example:

Not required

Rationale:

For correct execution of code and thereby functionality.

4.10.9. Rules_CtrlFlow_009

No hidden data flow or control flow except for call back function defined by SOFTWARE. "No hidden data flow or control flow" are meant to prohibit a control flow from processing multiple meanings. This is the rule to control the increase in risk of occurrence of problems by implementing a control which is not recognized by a designer.

Example:

Example of hidden control flow is shown below:

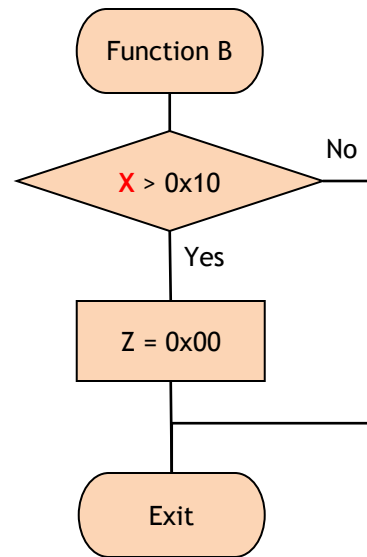
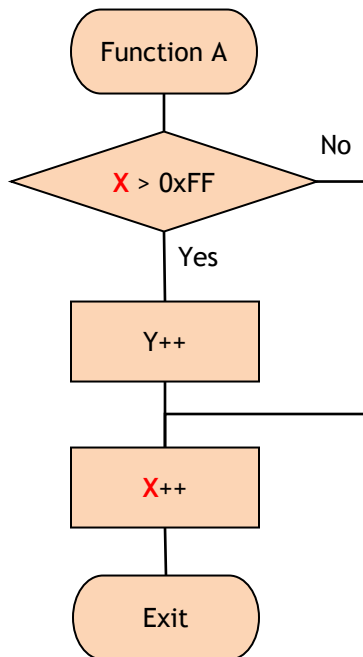
Variable 'X' in a function, created by an individual A, is used as a condition for assigning a variable 'Z' with no dependency within a function B, created by an individual B.

Preconditions:

- Global variables: X, Y.
- Function A and function B may be executed alternately.
- There is no dependency between variable "Z" in function B and global variable "X".

Function A created by an individual A:

Function B created by an individual B:
(Hidden control flow)



Rationale:

To avoid errors in implementation.

4.10.10. Rules_CtrlFlow_010

No unconditional jumps. Unconditional jumps like 'goto' shall not be used.

Example:

/ Not compliant */*

```

LOOP: do
{
    if (a == 15)
    {
        /* skip the iteration */
        a = a + 1;
        goto LOOP;
    }
    printf("value of a: %d\n", a);
    a++;
} while (a < 20);
  
```

/ Compliant */*

```

do
{
    if (a == 15)
    {
        /* skip the iteration */
        a = a + 1;
    }
    else
    {
        printf("value of a: %d\n", a);
    }
}
  
```

```
        a++;  
    }  
} while (a < 20);
```

Rationale:

To avoid errors in implementation.

4.10.11. Rules_CtrlFlow_011

When you use "switch", "while", "do-while", "for" statement, even if you can write one sentence, writing enclosed within the brace { }.

Example:

```
/* Not compliant */  
if(a == 15)  
    a = a + 1;
```

```
/* Compliant */  
if(a == 15) {  
    a = a + 1;  
}
```

Rationale:

To maintain the description of the consistency.

4.10.12. Rules_CtrlFlow_012

When use "if-else" or "switch". All conditions should be handled.

Example:

Not required.

Rationale:

To avoid errors in implementation.

4.10.13. Rules_CtrlFlow_013

There shall be no unreachable code.
e.g. delete the branch code by impossible conditions

Example:

Not required.

Rationale:

To avoid errors in implementation.

4.11. Functions

4.11.1. Rules_Func_001

Functions that do not return a parameter shall be declared as 'void' type.

Example:

```
/* Compliant */  
void Ctl_SystemCheck(void);
```

```
/* Not compliant */  
Ctl_SystemCheck(void);
```

Rationale:

Eliminates functions with the implicit return type of 'int' and improves readability.

4.11.2. Rules_Func_002

A function shall not return a pointer to a local object.

Example:

```
/* Not compliant */  
uint16 *Spi_GetSpi_Data(void)  
{  
    /* Data buffer */  
    uint16 t_Buffer[SPI_BUFFER_SIZE];  
    ...  
    /* update the buffer */  
    ...  
    return &t_Buffer[0];  
}
```

Rationale:

After return, the non-static local object no longer exists. Static objects will do, but they are defined at local scope to be sure that they are not accessed from outside a function.

4.11.3. Rules_Func_003

If a function returns a value, the calling code should check for an absence of errors.

Example:

Check that if the called function returns data have the expected range/value or not.

Rationale:

To ensure the called functionality is executed correctly.

4.11.4. Rules_Func_004

One entry and one exit point in subprograms and functions. The way to invoke one function should be same and consistent. The return for exit should be one in a subprogram and a function.

Example:

```

/* Not compliant */
unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    if(LuiVal1 > LuiVal2)
    {
        return LuiVal1;
    }
    else
    {
        return LuiVal2;
    }
}

```

```

/* Compliant */
unsigned int Compare(unsigned int LuiVal1, unsigned int LuiVal2)
{
    unsigned int LuiReturnValue = 0;
    if(LuiVal1 > LuiVal2)
    {
        LuiReturnValue = LuiVal1;
    }
    else
    {
        LuiReturnValue = LuiVal2;
    }
    return LuiReturnValue;
}

```

Rationale:

Readability, maintainability and avoidance of invalid operations.

4.11.5. Rules_Func_005

No recursions. Recursive functions shall not be used.

Example:

```

/* Not compliant */
void recurse()
{
    recurse();
}

```

Rationale:

There can be stack overflow, in case of recursive functions.

4.11.6. Rules_Func_006

All services and APIs of a module shall be multicore re-entrant.

Re-entrant definition (single core): A function is re-entrant if it can be interrupted somewhere in its execution and then safely called again before its previous invocation completes executing, produce correct results for both invocations.

Re-entrant definition (multiple cores): A function is re-entrant if it can be called on one core while it is already executing on any other core and still produce correct results for all calls.

Consequence:

- Must not use any static/global variable without resource protection (lock) if data consistency is required. Accesses from services (APIs) or processes or especially interrupt routines have to be considered.
- No assumptions about priorities and scheduling possible (e.g. for interrupt code).
- Must not modify its owned code.
- Must not call other non-re-entrant function(s).

Example:

1)

```
/* Compliant */
sint32 foo(sint32 b)
{
    return (b + 2);
}
sint32 bar(sint32 a)
{
    return (foo(a) * 10);
}
```

/* Not compliant */

```
sint32 a = 1;
sint32 foo(void)
{
    a += 2;
    return a;
}
sint32 bar(void)
{
    return (foo() * 10);
}
```

2)

```
/* Compliant */
sint32 a = 1;
sint32 foo(void)
{
    sint32 l_a;
    SCHM_ENTER_<Module>_<Res_a>();
    a += 2;
    l_a = a;
    SCHM_EXIT_<Module>_<Res_a>();
    return l_a;
}
```

/* Not compliant */

```
sint32 a = 1;
sint32 foo(void)
{
    SCHM_ENTER_<Module>_<Res_a>();
    a += 2;
    SCHM_EXIT_<Module>_<Res_a>();
    /* Value can be modified concurrently */
}
```

```
    return a;
}
```

Rationale:

To avoid errors in execution.

4.11.7. Rules_Func_007

All scheduled functions of a module shall be multicore re-entrant.

Example:

Not required

Rationale:

The re-entrancy is needed if the functions are scheduled in different tasks or on different cores.

4.12. Pointers and Arrays

4.12.1. Rules_Ptr_001

Limited use of pointers. Avoid abuse of pointers for unnecessary cases and consider design to avoid usage of pointers.

Example:

Just use one global pointer to point to value of configuration in configuration files ('<Msn>_Cfg.c', '<Msn>_PBcfg.c'). Don't or limit using local pointer that refers to structures in global configuration pointer. E.g.

- ICU module just uses pointer 'Icu_pCfgPtr'
- ADC module just uses pointer 'Adc_pCfgPtr'

Rationale:

To avoid errors in implementation.

4.12.2. Rules_Ptr_002

A structure as parameter for a function should be passed as pointer.

* Note: Consider that the content of the structure shall be stable until the function call is finished. The function could be interrupted and the values of the structure could be modified (side effects). If this constraint is not fulfilled other mechanisms (semaphores, disable interrupts, etc.) have to be used to ensure the intended behaviours. In all other cases, this rule helps to write efficient code.

Example:

```
/* Compliant */
/* Typedef in a header file */
typedef struct
{
    uint8 x;
    uint8 y;
    uint8 z;
} MyModule_Vector_tst;
```

```

/* Prototype of a function */
uint8 MyModule_VectorGetLength(const MyModule_Vector_tst *vector);

/* Usage in a c file */
uint8 vectorlength;
MyModule_Vector_tst Vector1_st;

vectorlength = MyModule_VectorGetLength(&Vector1_st);

```

Rationale:

Pointer to a structure is the most efficient way to pass a structure to a function via parameter.

4.12.3. Rules_Ptr_003

Pointers shall be checked before being de-referenced.

Example:

```

void Mod_FunctionX(uint8 *p_Data1, uint8 *p_Data2)
{
    /* Compliant */
    if (p_Data1 != NULL)
    {
        *p_Data1 = 0u;
    }

    /* Not compliant */
    *p_Data2 = 0u;
}

```

Rationale:

Avoid undefined behaviour if a pointer is NULL.

4.12.4. Rules_Ptr_004

A 'typedef' shall be used to simplify program syntax when declaring function pointers.

Example:

```

typedef uint8 (*Mod_FuncX_t)(void);
Mod_FuncX_t Mod_MyFunc;

```

Rationale:

Improved readability

4.12.5. Rules_Ptr_005

To address a structure's fields via a pointer to the structure, the notation '*Ptr->Field*' shall be used.

Example:

```

/* Not compliant */
if (0 == (*DataPtr).Field)

```


Rationale:

Improved readability

4.13. Structures and Unions

4.13.1. Rules_Struct_001

All members of a structure shall be named and shall only be accessed via their names.

Example:

```
typedef struct STag_Mod_Param
{
    uint8 ucIndex;
    uint32 ulValue;
} Mod_Param;

Mod_Param s_Param;

/* Compliant */
s_Param.ucIndex = 0u;

/* Not compliant */
*((uint8 *)&Mod_Param + 1) = 0u;
```

Rationale:

Avoid reading/writing to unnamed locations in memory.

4.14. Pre-processing Directives

4.14.1. Rules_PreProcess_001

Header files which are part of predefined program libraries shall be included using '<>'.
Header files which are part of the source code generated in the software project shall be included with '""' (double quotes symbols).

Example:

```
/* Compliant */
#include <string.h>
#include "Eep_Read.h"

/* Not compliant */
#include "string.h"
#include <Eep_Read.h>
```

Rationale:

Library and project files shall be easily identified.

4.14.2. Rules_PreProcess_002

Each header file shall protect itself against multiple inclusions.

Example:

```
/* Compliant */
#ifndef FILENAME_H
#define FILENAME_H
.....
#endif /* FILENAME_H */
```

Rationale:

Avoid multiple re-definitions.

4.14.3. Rules_PreProcess_003

Each module shall include its own header file.

Example:

Not required

Rationale:

This is the only way of allowing the compiler to check for consistency between declaration and definition of global variables and functions.

4.14.4. Rules_PreProcess_004

A file should include all the file they need and not rely on secondary includes (notice that only files which are really required shall be included).

Example:

Assume we have following requirement:

- 1) "A.h" requires element in "B.h" and "C.h"
- 2) "B.h" requires element in "C.h"

```
// Case I
A.h
|---- include B.h
|      |-----include C.h
|---- include C.h
// Case II
A.h
|---- include B.h
|      |-----include C.h
```

With requirement 1) and 2), both case is correct.

However, in case that "B.h" do not need the element from "C.h" anymore, we need to delete the "include C.h" inside "B.h". These above example will become:

// Case I => still ok, due to inside "A.h" already have 2 include commands of "B.h" and "C.h"

```
A.h
|---- include B.h
|---- include C.h
```

// Case II => error will occur due to in the pass, "A.h" is depended on "B.h" to refer to element in "C.h"

```
A.h
|---- include B.h
```

|----- Missing "C.h"

In final conclusion, "Case I" is recommended example for this rule.

Rationale:

To avoid unnecessary dependencies.

4.14.5. Rules_PreProcess_005

Path shall not be specified (absolute or relative) for the declaration of the included file.

Example:

```
/* Compliant */
#include "Nm_Gbl.h"

/* Not compliant */
#include "c:\gnm\Nm_Gbl.h"
```

Rationale:

The path may be different in different environments.

4.14.6. Rules_PreProcess_006

Direct values (fixed value, addresses, etc.) should not be hard-coded in the source code. Macro ('*#define*', '*constant*', etc.) should be used preferably.

Example:

```
/* Compliant */
#define TIMEOUT 4
...
if (timerValue >= TIMEOUT)
```

Rationale:

The values of the constants can be changed if required at one place, instead of changing everywhere where referred and avoid making errors.

4.14.7. Rules_PreProcess_007

Only the following pre-processor directives shall be used: '*#if*', '*#elif*', '*#else*', '*#endif*', '*#ifdef*', '*#ifndef*', '*#define*', '*#include*', '*#error*' and '*#pragma*'.

Example:

Not required.

Rationale:

To avoid using compiler specific pre-processor directives.

4.14.8. Rules_PreProcess_008

Compiler switches shall be compared with defined values. Simple checks if a compiler switch is defined shall not be used. In general 'ON' and 'OFF' shall be used to switch functionality on or off. These symbols and their values are defined in a header file which define common standard types.

Example:

```
/* Compliant */
#if (EEP_DEV_ERROR_DETECT == ON)

/* Not compliant */
/* In the header file */
#define EEP_DEV_ERROR_DETECT OFF
/* In the source file */
#ifdef EEP_DEV_ERROR_DETECT
```

Rationale:

To avoid errors in implementation, if the defined value for a compiler switch is not intended.

4.14.9. Rules_PreProcess_009

Macros used in place of functions should have the same notation as a function.

Example:

```
#define Wdg_Trigger() (WD_TIM = 0xff)
```

Rationale:

Not required

4.14.10. Rules_PreProcess_010

If a macro definition represents several statements, the entire statement shall be encapsulated within curly brackets. For multi-lines macro definitions use the '\' (backslash or yen) symbol at the end of line. E.g.

```
{ \
    Statement 1; \
    Statement 2; \
}
```

Example:

```
#define Swap(x,y,h) \
{ \
    h = x; \
    x = y; \
    y = h; \
}
```

Rationale:

Readability

4.14.11. Rules_PreProcess_011

In order to avoid hidden unintended type conversions, the explicit casting in macros for macro parameters shall not be allowed. If type casting is needed, it shall be done explicitly by the macro caller. Exceptions shall be documented.

Examples:

```
/* Not compliant */
/* A 32 bits value parameter will be implicitly converted to 'uint16' */
#define WRITE_REGISTER_16BIT(address, value) \
    ((* (volatile uint16*) (address)) = (uint16) (value))

/* Compliant */
/* It will detect value parameters which are not 'uint16' */
/* Address casting is tolerated as an exception for the memory mapped I/O access */
#define WIRTE_REGISTER_16BIT(address, value) \
    ((* (volatile uint16*) (address)) = (value))
```

Rationale:

The type of parameter need be checked correctly by user. Avoid the incorrect value in parameter input.

4.14.12. Rules_PreProcess_012

Macros shall not use global names, since the global names may be hidden by a local declaration.

Example:

```
/* Not compliant */
/* Adc_GucStatus is global variable holds current status of ADC driver */
#define CHANGE_STATUS(x) {Adc_GucStatus = (x)}
```

Rationale:

To avoid hidden data flow.

4.15. Optimization

4.15.1. Rules_Opt_001

Eliminate unnecessary recalculations of values.

Example:

Not required

Rationale:

Performance

4.15.2. Rules_Opt_002

Combine equivalent constants, however, this is at the cost of readability. Appropriate comments should be provided where necessary.

Example:

```
/* Compliant */
A = B | CONST1;
A = A & CONST2;
/* Can be */
A = ((B | CONST1) & CONST2)
```

Rationale:

Performance

4.15.3. Rules_Opt_003

The condition of most probable occurrence should be placed in the *'if'* rather than in the *'else'*.

Example:

Not required

Rationale:

Performance

4.15.4. Rules_Opt_004

Substitute operations to conserve resources. Optimize *'|'* (*or*) and *'&'* (*and*) expressions: An *'|'&'* query should be held as short as possible:

- At an *'|'* query, put the most probable case in the first statement.
- At an *'&'* query, put the most improbable case in the first statement.
- For the query of the same variables use a mask.

Example:

/ Given that variable 'stNxt' and all macros in below example, each has a value with either all bits turned off or only 1 bit turned on. */*

```
/* Compliant */
/* Optimization with a mask */
#define SYNC_ALL_BITS (SYNC_TIMEOUT | SYNC_WAIT_INC | SYNC_RESYNC_OFFSET |
SYNC_PHASE_PLAUS_CHK | SYNC_PHASE_PLAUS_SECOND_CHK)

if ((stNxt & SYNC_ALL_BITS) != 0)
{
    ...
}

/* Not compliant */
/* CPU has to go through all conditions to check if any of them is true */
if ((stNxt == SYNC_TIMEOUT) || (stNxt == SYNC_WAIT_INC) || (stNxt ==
SYNC_RESYNC_OFFSET) || (stNxt == SYNC_PHASE_PLAUS_CHK) || (stNxt ==
SYNC_PHASE_PLAUS_SECOND_CHK))
{
    ...
}
```

Rationale:

Performance

4.15.5. Rules_Opt_005

The time for the execution of interrupt service routine should be minimal. The delay in case of nested interrupts is to be documented.

Example:

Not required

Rationale:

Performance

4.15.6. Rules_Opt_006

Pre-processor macros/configuration data/source code shall be memory mapped to the correct section.

Example:

Not required.

Rationale:

To avoid problems when memory protection is used.

4.15.7. Rules_Opt_007

Memory mapping shall not be used for variables defined within a function. Memory mapping header files shall not be included inside the body of a function.

Example:

```
/* Compliant */
void process(void)
{
    /* No memory mapping needed */
    uint32 my_counting_var_u32;
}
```

Rationale:

It is impossible to map the local variables in a function to a memory section.

4.15.8. Rules_Opt_008

Avoid wastage of bytes by aligning variables in memory:

- 16-bit variables have to be located at an address divisible by 2.
- 32-bit variables have to be located at an address divisible by 4.

It is recommended to sort by size (e.g. first all pointers, then all 32-bit variables, then all 16-bit variables and then all 8-bit variables).

* Note: Pointers are 32-bit sized (also uint8*)

Example:

1)

/* Compliant */

```
uint16* XYZ_adrData_pul6;  
uint16 XYZ_stMachine1_ul6;  
uint8 XYZ_xTestval1_u8;  
uint8 XYZ_xTestval2_u8;
```

/* Not compliant */

```
uint8 XYZ_xTestval1_u8;  
/* Gap 3 bytes */  
uint16* XYZ_adrData_pul6;  
uint8 XYZ_xTestval2_u8;  
/* Gap 1 byte */  
uint16 XYZ_stMachine1_ul6;
```

2)

/* Compliant */

```
typedef struct  
{  
    uint32 a;  
    uint32 c;  
    uint8 b;  
    uint8 d;  
} st1; /* This structure occupies only (2*4+2*1) bytes of memory */
```

/* Not compliant */

```
typedef struct  
{  
    uint32 a;  
    uint8 b;  
    uint32 c;  
    uint8 d;  
} st1; /* This structure occupies (4*4) bytes of memory */
```

Rationale:

Memory optimization